თბილისის თავისუფალი უნივერსიტეტი

კომპიუტერული მეცნიერებების, მათემატიკისა და ინჟინერიის სკოლა (MACS[E])

ელექტრო და კომპიუტერული ინჟინერიის პროგრამა

ბეჟანიშვილი ნიკოლოზ
ცხვედაძე ქეთევანი

ელექტრო და კომპიუტერული ინჟინერიის ლაბორატორია - პროექტის ანგარიში
თვითბალანსირებადი რობოტი

თბილისი

2026

Annotation

This project was completed as part of the “Electrical and Computer Engineering Laboratory” course. The objective was to design and build a two-wheeled self-balancing robot - a physical implementation of the inverted pendulum control problem.

The robot uses an MPU6050 six-axis IMU to measure tilt. A complementary filter fuses the accelerometer and gyroscope readings to estimate the lean angle at 200 Hz. A PID controller converts the angle error into motor commands delivered to two motored wheels. The firmware is organized as three concurrent RTOS tasks running across the ESP32’s two cores, separating the hard-real-time balance loop from soft-real-time networking and sensor tasks. Additional features include HC-SR04 ultrasonic distance sensing, battery voltage monitoring, and NVS-based persistent gain storage. A WiFi access point exposes a web interface for live angle telemetry, real-time gain tuning (Kp, Ki, Kd, offset), remote drive control, and battery voltage display. All primary objectives were achieved: the robot balances stably and all parameters can be tuned in real time without rebooting.

ანოტაცია

ეს პროექტი შესრულდა კურსის „ელექტრო და კომპიუტერული ინჟინერიის ლაბორატორია“ ფარგლებში. პროექტის მიზანი იყო ორბორბლიანი თვითბალანსირებადი რობოტის შექმნა.

რობოტი იყენებს MPU6050 ექვსღერძიან IMU-ს დახრის გასაზომად. კომპლემენტარული ფილტრი აერთიანებს აქსელერომეტრისა და გიროსკოპის მონაცემებს 200Hz სიხშირით დახრის კუთხის შესაფასებლად. PID კონტროლერი კუთხის გადახრის საფუძველზე ატრიალებს ძრავებს ბალანსის შენარჩუნებლად. პროგრამა სამ პარალელურ RTOS Task-ად არის ორგანიზებული. WiFi წვდომის წერტილი უცრუნველყოფს ვებ-ინტერფეისს ტელემეტრიით, PID-ის პარამეტრების რეალურ დროში დარეგულირების შესაძლებლობით და დისტანციური მართვით. ყველა მთავარი მიზანი მიღწეული იქნა.

Table Of Contents

Table of Figures

Introduction

This project was completed as the self-balancing robot assignment for the Electrical and Computer Engineering Laboratory course at Tbilisi Free University. The goal was to design, build, and program a two-wheeled self-balancing robot that demonstrates key embedded systems concepts: sensor fusion, closed-loop control, real-time scheduling, and wireless human-machine interface design.

The robot is mechanically analogous to a Segway: two driven wheels on a common axle support a vertical chassis. Without active control, the structure is unstable and will topple. Balance is maintained by continuously reading the tilt angle from an MPU6050 inertial measurement unit (IMU), fusing accelerometer and gyroscope measurements through a complementary filter at 200 Hz, and driving TT gearmotors through a TB6612FNG dual H-bridge in the direction that corrects the lean. The control law is a PID (Proportional-Integral-Derivative) controller.

The firmware runs on an ESP32 microcontroller using the FreeRTOS real-time operating system, structured as three concurrent tasks with explicit priority and core affinity. The balance-critical task is isolated on one CPU core; networking and ranging tasks run on the other. This prevents any soft-real-time activity from disturbing the 5ms control interval.

The robot exposes a WiFi access point. Connecting a phone or laptop to it opens a web dashboard that shows a live angle graph, the current ultrasonic distance reading, battery voltage, and sliders for all tunable parameters. Gains can be saved to ESP32 NVS flash so they persist across power cycles. The chassis was designed in CAD and produced using 3D printing and laser-cut plywood.

The project had the following requirements:

- **Correct Scheme / Wiring**: Correctly wiring ESP32, MPU6050, Driver and batteries are correctly wired.
- **Frame and CAD Design**: The robot has a systemic construction; the center of mass is correctly placed and a CAD design of the robot is present.

- **Sensor Reading**: Reliably reading calibrated values from the MPU6050
- **Filtering and Calculating Angle:** Filtering IMU data and reliably calculating the lean angle.
- **PID Controller and Tuning**: The robot holds balance when turned on and can restore balance after a small push.
- **Radio Control**: Remote control using ESP32's WiFi/Bluetooth, stable connection and control.
- **Object Detection and Avoidance:** Detecting object in front using a distance measuring sensor and avoiding them despite radio commands.
- **Telemetry:** The remote controller displays the battery charge level, current lean angle and RSSI WiFi/Bluetooth signal level in real-time.

An additional requirement is that object avoidance and telemetry should run on a separate thread independent of the control loop using RTOS.

Technical Review

Physical Design

The chassis is a combination of 3D-printed structural parts and laser-cut plywood for outer frame. CAD files and G-code are included in the repository under the CAD Files/, 3D Printing/, and Laser/ subdirectories. The two TT gearmotors are mounted symmetrically on either side of the lower chassis plate. We mounted MPU6050 IMU between the wheels in a vertical orientation aligned with the robot's axis of balance to minimize the error caused by the centripetal and tangential accelerations. We positioned the battery pack at the top of the chassis. This placement is intentional: raising the center of mass makes the robot tip more slowly, giving the PID loop more time to react to disturbances. The power system uses two Li-ion batteries (3.7V nominal, 4.2V max) wired in series via BMS. The voltage is stepped down to 5 V via a buck converter for ESP and the ultrasonic sensor. The ESP32's on-board 3.3 V linear regulator powers the TB6612FNG logic supply (VCC) and the MPU6050.

We designed the frame in a way that allows for easy assembly using box joints. There are dedicated holes for wires, switch, motor axis, etc. The files for each piece are stored as individual parts, the full outer frame of the bot uses these parts and is stored as an assembly and the complete assembly of the bot includes the frame as well as other hardware. This allows for easier file management and simplifies making changing due to the architecture of SolidWorks.

IMU & Angle Estimation

The MPU6050 (GY-521 breakout) is a six-axis IMU providing a three-axis accelerometer and three-axis gyroscope over I2C. It is powered by the ESP32's 3.3 V regulator with SDA on GPIO 21 and SCL on GPIO 22. The AD0 pin is tied to GND, setting the I2C address to 0x68. The INT, XDA, and XCL pins are left unconnected.

On startup, a gyroscope calibration routine runs while the robot is held stationary. It takes a fixed number of readings and computes the mean gyroscope output on each axis. These offsets are subtracted from all subsequent gyro readings, removing static drift bias.

Angle estimation uses a complementary filter. The accelerometer provides an absolute tilt angle derived from the gravity vector projection, but is noisy due to vibration and motor-induced acceleration. The gyroscope provides a smooth rate signal that integrates to angle, but accumulates drift over time. The complementary filter combines both at a fixed 5 ms interval:

$$angle = \alpha * (angle + gyro_{rate} * dt) + (1 - \alpha) * accel\_angle$$

Where $\alpha = 0.98$ is the high-pass weight on the gyro path and $dt = 5\ ms$. The filter trusts the gyroscope for fast dynamics and the accelerometer for slow drift correction. The controlTask uses vTaskDelayUntil, scheduling each wake-up on a fixed absolute deadline so that dt is exactly constant regardless of how long the sensor read and math take

PID Balance Controller

The PID controller runs every 5 ms in controlTask. It computes a motor output from the angle error as follows:

$$error = measured\_angle - setpoint$$

$$integral += error \times dt$$

$$output = Kp * error + Ki * integral + Kd * \frac{error - prev_{error}}{dt}$$

The setpoint (balance offset) compensates for asymmetric component placement where the robot's true center of mass does not coincide with the geometric axis. It is calibrated by holding the robot near vertical, reading the stabilized angle from the web interface, and entering that value as the offset.

TT gearmotors have a significant stiction deadband: small PWM commands below a threshold produce no movement due to motor friction and gear backlash. The deadband compensation adds the configured minimum PWM value to any non-zero output before sending it to the driver, ensuring that even small corrective commands produce actual wheel movement. All gains, the balance offset, and the deadband value are stored in the mutex-protected shared State struct and can be changed via the web interface with immediate effect.

WiFi Control & Telemetry Interface

The ESP32 operates as a WiFi access point (SSID: BalanceBot, password: balance123), requiring no external router. ESPAsyncWebServer and AsyncTCP serve a single-page HTML/JavaScript dashboard at 192.168.4.1 over WebSocket.

The dashboard features a live angle graph updated in real time, the latest ultrasonic distance reading, battery voltage, sliders for Kp, Ki, Kd, balance offset, and motor deadband, a GO/STOP button to arm or disarm the balance loop, a Save to Flash button that writes current gains to ESP32 NVS for persistence across power cycles, and forward/backward drive buttons that temporarily shift the balance setpoint to command the robot to lean and move.

The commsTask runs at 25 Hz on Core 0, pushing JSON telemetry snapshots to all connected WebSocket clients. The mutex is held only briefly to copy the current state into local variables, then released before the WebSocket send, keeping the critical section short.

We used a resistor voltage divider to step the 8.4V battery voltage down into the 0–3.3 V range readable by the ESP32 ADC. GPIO 35 is used as the ADC input because it is a dedicated input-only pin with no alternate function conflicts, making it ideal for analog sensing. The divider is calculated so that the maximum expected battery voltage (nominally 8.4 V for a two-cell lithium pack at full charge) maps to approximately 3.1 V at the ADC input, staying within the 3.3 V rail with margin. The ADC reading is converted back to battery voltage by the inverse of the divider ratio and sent to the web dashboard as part of the commsTask telemetry. We found the values of 47kΩ and 100kΩ to work best for our purposes.

Distance Detection & Object Avoidance

We used the HC-SR04 Ultrasonic sensor for distance measurement. It measures distance by emitting a 10 μs pulse on the TRIG pin and timing the return echo on the ECHO pin. Distance is calculated as:

$$distance = duration \times \frac{0.034}{2}$$

giving the result in centimetres. A pulseIn timeout of 30,000 μs prevents indefinite blocking when no echo returns.

The HC-SR04 ECHO pin outputs 5V logic. GPIO 25 is protected by a 1 kΩ / 2 kΩ resistor voltage divider that scales the 5 V echo signal down to approximately 3.3 V before it reaches the ESP32 input. The pulseIn() call is blocking and can stall for up to 30 ms. In a monolithic loop design this would destroy the 200 Hz control timing. The solution is to isolate ranging in a dedicated FreeRTOS task (distanceTask) running at 10 Hz on Core 0 at low priority. The measured distance is stored in the shared State struct under the mutex, and the control task reads only the cached value without ever blocking

FreeRTOS Firmware Architecture

The ESP32 Arduino core runs on FreeRTOS. Rather than a monolithic loop(), the firmware is organized into three explicit tasks with defined priorities and core affinity, separating the one hard-real-time activity (balance control) from soft-real-time activities (networking, ranging):

Task	Core	Priority	Rate	Responsibility
controlTask	1	3 (high)	200 Hz	MPU read → complementary filter → PID → motor output
commsTask	0	1	25 Hz	JSON telemetry push to WebSocket clients
distanceTask	0	1	10 Hz	HC-SR04 ranging (blocking pulseIn call)

Deterministic control timing is achieved by vTaskDelayUntil in controlTask, which schedules each wake-up on an absolute tick deadline. This gives a constant $dt = 5ms$ regardless of how long the IMU read and PID math take. Core isolation pins controlTask alone to Core 1; nothing on Core 0 can preempt the balance loop.

All shared data lives in a single State struct: the gain values, current angle, armed/run flag, and ultrasonic distance. The struct is protected by a FreeRTOS mutex (stMutex). Each task acquires the mutex only long enough to snapshot values into local variables, then releases it before any slow operation (motor writes, WebSocket sends, pulseIn). This keeps critical sections short and eliminates nested locking, making deadlock impossible.

Conclusion

All requirements were met. The robot maintains balance using a complementary filter angle estimate feeding a PID controller, with all parameters accessible and tunable in real time through a WiFi web interface without reflashing.

Several engineering challenges were encountered and addressed during development. Gyroscope drift was mitigated by the startup calibration routine and the complementary filter's low-frequency correction from the accelerometer. The blocking nature of pulseIn() was addressed by isolating ranging in a dedicated low-priority RTOS task so it cannot affect control timing.

The primary hardware limitation of the system is the TT gearmotors. These motors have a noticeable stiction deadband, gear backlash, and limited maximum speed. The deadband compensation in the firmware partially addresses the stiction, but gear backlash introduces a mechanical hysteresis that the control loop cannot fully compensate. Encoderless operation means there is no closed-loop speed or position feedback at the motor level, which sets a ceiling on achievable balance quality regardless of PID tuning.

The most valuable learning outcome was the end-to-end design of a real-time embedded system: understanding how blocking I/O operations must be isolated from time-critical loops, how shared state must be protected with mutexes in concurrent firmware, and how physical phenomena (center-of-mass placement, motor stiction, ADC scaling) must be accounted for at both the hardware and firmware level. The project also provided practical PID tuning experience, where the interaction between Kp, Kd, and the mechanical deadband became directly observable through the live web interface.

Skills and Technologies

- Visual Studio Code (with PlatformIO for ESP programming)
- SolidWorks for CAD design.
- CorelDRAW for editing vector cut designs.
- Programming/markup languages: C++, HTML.
- ESP32 microcontroller programming (C++, Arduino framework, PlatformIO)
- Real-time system design with WebSocket.
- ESPAsyncWebServer and AsyncTCP for WiFi access point and WebSocket server.
- NVS (Non-Volatile Storage) for persistent parameter storage across power cycles.
- FreeRTOS: multi-task firmware design, task priorities, core affinity, vTaskDelayUntil.
- Mutex synchronization for shared state in concurrent firmware
- I2C communication protocol for MPU6050.
- Complementary filter design for IMU sensor fusion
- PID controller design, implementation, and empirical tuning
- Laser Cutting machine competence for cutting out and engraving various designs
- Cable management and labeling, designing maintainable and easily replicable modular systems.
- Physical assembly skills: Soldering, wood cutting & gluing
- Git version control and GitHub repository management with meaningful commit conventions
- Hardware debugging: multimeter use, serial monitor diagnostics, pin conflict resolution

Individual contributions

Nikoloz Bezhanishvili:

- Mechanical chassis design using SolidWorks
- 3D printing and laser cutting
- Physical assembly, component mounting, and cable management
- Motor mounting and mechanical alignment
- Hardware soldering and wiring
- Project testing, PID tuning, and demonstration preparation
- GitHub repository setup and commit management
- Research
- Project report writing

Ketevani Tskhvedadze:

- Full ESP32 firmware architecture (FreeRTOS task structure, shared State, mutex)
- MPU6050 driver, gyroscope calibration, and complementary filter implementation
- PID controller design and implementation with deadband compensation
- ESPAsyncWebServer WiFi access point and web dashboard (live graph, sliders, drive control)
- HC-SR04 object detection and avoidance (distanceTask, voltage-divider level shift)
- Battery voltage monitoring circuit (GPIO 35 divider) and firmware integration
- NVS gain persistence implementation
- Hardware soldering and wiring, sensor integration, and debugging
- Research
- Project report writing

GitHub repository

All source code, CAD files, 3D printing G-code, and laser-cut files for this project are publicly available on GitHub. The repository is organized into the following subdirectories:

- self-balancing-code/ — PlatformIO project with full C++ firmware
- CAD Files/ — CAD design files for the chassis
- 3D Printing/ — G-code files for 3D-printed structural parts
- Laser/ — Files for laser-cut flat panels

Repository: https://github.com/nbezh/ECE-self-balancing-robot

Contributors' profiles:

Nikoloz Bezhanishvili – https://github.com/nbezh

Ketevani Tskhvedadze - https://github.com/tskhvedadzee

References

Datasheets & Component Information:

[1] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification,” Datasheet. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[2] Toshiba Semiconductor, “TB6612FNG Dual DC Motor Driver IC,” Datasheet. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/209390/TOSHIBA/TB6612FNG.html.

[3] Multicomp, “HC-SR04 Ultrasonic Ranging Module Datasheet,” Alldatasheet. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/2216428/MULTICOMP/HC-SR04.html.

[4] Espressif Systems, ESP32 Technical Reference Manual. [Online]. Available: https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf.

Software & Frameworks:

[5] FreeRTOS, “FreeRTOS Kernel Documentation,” FreeRTOS.org. [Online]. Available: https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals.

[6] ESP32Async, “ESPAsyncWebServer,” GitHub. [Online]. Available: https://github.com/ESP32Async/ESPAsyncWebServer.

[7] “PlatformIO Latest Documentation,” PlatformIO. [Online]. Available: https://docs.platformio.org/en/latest/.

Articles & Resources:

[8] Espressif Systems, “Non-Volatile Storage (NVS) Library,” ESP-IDF Documentation. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html.

GrabCAD 3D Models:

[9] "Perfboard," GrabCAD. [Online]. Available: https://grabcad.com/library/perfboard-1.

[10] "ESP32 DevKit V1 3D Model," GrabCAD. [Online]. Available: https://grabcad.com/library/esp32-devkit-v1-3d-model-1.

[11] "TB6612FNG Driver," GrabCAD. [Online]. Available: https://grabcad.com/library/tb6612fng-driver-1.

[12] "MPU6050 Accelerometer Module," GrabCAD. [Online]. Available: https://grabcad.com/library/mpu6050-accelerometer-module-1.

[13] "HC-SR04 Ultrasonic Sensor," GrabCAD. [Online]. Available: https://grabcad.com/library/hc-sr04-ultrasonic-sensor-10.

[14] "Boitier 2 Accus 18650 / 2 Slots 18650 Battery Holder," GrabCAD. [Online]. Available: https://grabcad.com/library/boitier-2-accus-18650-2-slots-18650-battery-holder-1.

[15] "Generic 18650 Battery," GrabCAD. [Online]. Available: https://grabcad.com/library/generic-18650-battery-2.

[16] "TT Motor Dual Shaft," GrabCAD. [Online]. Available: https://grabcad.com/library/tt-motor-dual-shaft-1.

[17] "TT Motor Wheel Yellow," GrabCAD. [Online]. Available: https://grabcad.com/library/tt-motor-wheel-yellow-1.

[18] "Switch On Off White," GrabCAD. [Online]. Available: https://grabcad.com/library/switch-on-off-white-1.

Attachments & Project Visuals

Table 1: ESP connections

GPIO	Board Label	Connected To	Module	Voltage
4	D4	TB6612 PWMA	Motor Driver	3.3V out (PWM)
5	D5	TB6612 STBY + 10 kΩ to GND	Motor Driver	3.3V out
16	D16	TB6612 AIN2	Motor Driver	3.3V out
17	D17	TB6612 AIN1	Motor Driver	3.3V out
18	D18	TB6612 BIN1	Motor Driver	3.3V out
19	D19	TB6612 BIN2	Motor Driver	3.3V out
23	D23	TB6612 PWMB	Motor Driver	3.3V out (PWM)
21	D21	MPU6050 SDA	IMU	3.3V (I2C)
22	D22	MPU6050 SCL	IMU	3.3V (I2C)
26	D26	HC-SR04 TRIG	Distance	3.3V out
25	D25	HC-SR04 ECHO via 1 kΩ/2 kΩ divider	Distance	3.3V in
35	D35 (input only)	Battery voltage divider output	Power Monitor	3.3V in (ADC)
3V3	3V3	MPU6050 VCC, TB6612 VCC	Power	3.3V source
VIN	VIN	TB6612 VM, HC-SR04 VCC	Power	5V source
GND	GND	All component GNDs	Power	Ground

Table 2: MPU6050 IMU connections

Pin/Signal	ESP32 GPIO	Notes
VCC	3V3	3.3V supply. MUST be 3.3V; 5V will damage the module
GND	GND	Common ground
SCL	GPIO 22	I2C clock
SDA	GPIO 21	I2C data
AD0	GND	Sets I2C address to 0x68
INT / XDA / XCL	Leave unconnected	Not used in this project

Table 3: TB6612FNG motor driver connections

Pin/Signal	ESP32 GPIO	Notes
PWMA	GPIO 4	Motor A speed (PWM)
AIN2	GPIO 16	Motor A direction
AIN1	GPIO 17	Motor A direction
STBY	GPIO 5	Driver enable. 10 kΩ pulldown to GND keeps LOW during boot
BIN1	GPIO 18	Motor B direction
BIN2	GPIO 19	Motor B direction
PWMB	GPIO 23	Motor B speed (PWM)
GND	GND	Common ground
PWMA	GPIO 4	Motor A speed (PWM)
AIN2	GPIO 16	Motor A direction
VM	VIN (5 V)	Motor supply from battery pack
VCC	3V3	Logic supply from ESP32 3.3V regulator
AO1/AO2	—	Left motor
BO1/BO2	—	Right motor

Table 4: HC-SR04 ultrasonic sensor connections

Pin/Signal	ESP32 GPIO	Notes
VCC	VIN (5 V)	5V required from buck converter
GND	GND	Common ground
TRIG	GPIO 26	Output: triggers 10 µs ultrasonic pulse
ECHO	GPIO 25 via 1 kΩ/2 kΩ divider	5V output scaled to ≈3.3V before ESP32

Table 5: Hand-tuned values

Name	Value	Notes
PID Kp	34	One leg to 3.3V
PID Ki	59	Other leg to GPIO 33 AND 10kΩ to GND
PID Kd	0.5	Voltage divider. Required
PID offset (deg)	4.3	Digital output. HIGH when motion detected
Min PWM (deadband)	48	5V required
Voltage divider for Battery reading	100 kΩ / 47kΩ	Common ground

Table 6: Voltage connections

Voltage	Source	Components Powered	Notes
8 V	Battery pack	Input to buck converter, TB6612 VM	Two-cell lithium: 6.4–8.4V range
5 V	Buck converter output	ESP32 VIN, HC-SR04 VCC	Set to exactly 5.0V. 470–1000 µF bulk cap on rail
3.3 V	ESP32 on-board reg.	TB6612 VCC, MPU6050 VCC, GPIO logic	Max ≈500 mA total from ESP32 regulator
			Notes
8 V	Battery pack	Input to buck converter, TB6612 VM	Two-cell lithium: 6.4–8.4V range
5 V	Buck converter output	ESP32 VIN, HC-SR04 VCC	Set to exactly 5.0V. 470–1000 µF bulk cap on rail

Figure 1: Frontal design

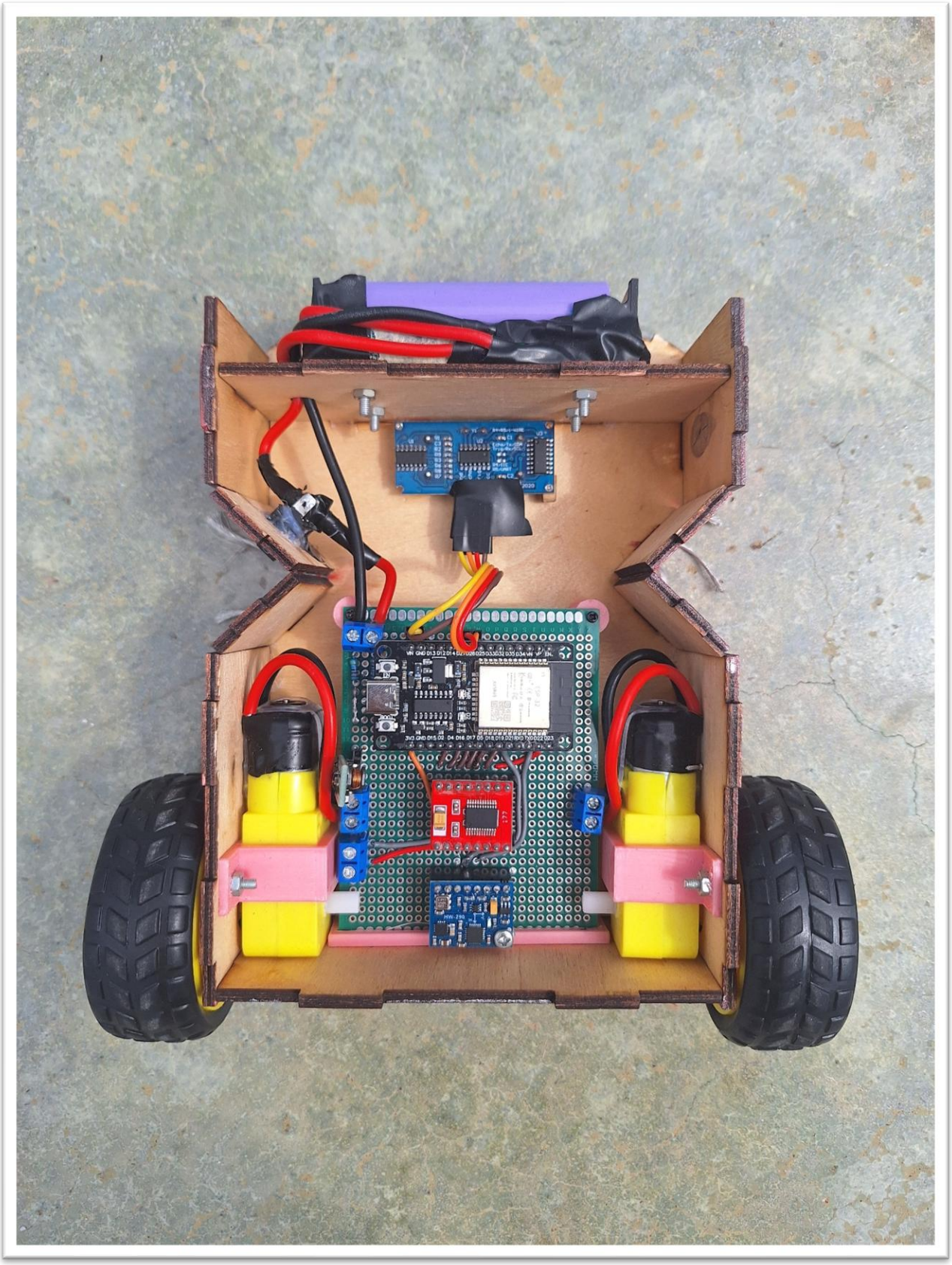

Figure 2: Internal design & wiring